2. a. For this assignment, I used Claude and ChatGPT to generate the code.

b. I accessed the code by using Claude and ChatGPT on my laptop.

c. Write a C program that picks a secret number between 1 and 10.

The user gets up to 3 tries to guess the number.

Requirements

• Use a fixed secret number (for example, 7).

• Use printf to prompt the user and scanf to read their

guess.

• Use a while or for loop to let the user guess up to 3 times.

• Use if-else statements to check if the guess is too high,

too low, or correct.

• Stop the loop immediately if the user guesses the correct

number.

• Print a final win or lose message.

This is the prompt I put in. I also put the output text so that the output is known to the AI.

d.

Claude

```
#include <stdio.h>

int main(void) {
    int secret = 7;
    int guess;
    int won = 0;

    printf("Guess a number between 1 and 10.\n");

    for (int attempt = 1; attempt <= 3; attempt++) {
        printf("Attempt %d/3. Enter your guess: ", attempt);
        scanf("%d", &guess);
```

```
        if (guess == secret) {
            printf("Correct! You win!\n");
            won = 1;
            break;
        } else if (guess < secret) {
            printf("Too low! Try again.\n");
        } else {
            printf("Too high! Try again.\n");
        }
    }

    if (!won) {
        printf("Out of tries! The number was %d. You lose.\n", secret);
    }

    return 0;
}
```

ChatGPT

```
#include <stdio.h>

int main() {
    int secret = 7;
    int guess;
    int i;

    printf("Guess a number between 1 and 10.\n");

    for (i = 1; i <= 3; i++) {
        printf("Attempt %d/3. Enter your guess: ", i);
        scanf("%d", &guess);

        if (guess == secret) {
            printf("Correct! You win!\n");
            break;
        }
        else if (guess < secret) {
            printf("Too low! Try again.\n");
        }
        else {
            printf("Too high! Try again.\n");
        }
    }

    if (guess != secret) {
        printf("Sorry! You lose. The secret number was %d.\n", secret);
    }
```

```
    return 0;
}
```

e. Correctness- Both programs correctly implement the number guessing game and allow the user up to three attempts. Both provide appropriate feedback. Claude's code is a bit better because it tracks when the user 'won' with the won variable, while chatgpts code checks the final value of 'guess' instead. Both programs work correctly for the correct integer inputs, but neither can handle invalid input such as letters.

Execution time- I believe both have the same time complexity because the loop executes a maximum of three times. Neither program has a time advantage over the other. The break statement in both programs ends the loop early when the user guesses correctly, which will avoid unnecessary iterations.

Space complexity: Both programs have O(1) space complexity. Claude's program uses one additional integer variable (won), so it technically uses slightly more memory, but this does not affect the overall space complexity. They do not use arrays, data structures that increase with the input size, so neither of them really affects space complexity.

Maintainability- Claude's code is slightly more maintainable because it uses descriptive names such as 'attempt' and 'won'. ChatGPT uses i, which is less descriptive. Claude uses a "won" variable, which makes the program's logic easier to understand and modify. However, both programs are relatively simple and easy to maintain, but neither can handle invalid inputs.

f. I selected Claude's program for my assignment because it has clear variable names and tracks the player's win status using the 'won' variable. Although both programs have the same time complexity and space complexity, Claude's code is easier to maintain, read, understand, and modify.

g. Correctness- I corrected the invalid input error without terminating the program. Invalid inputs count as an attempt, and the user still has the remaining attempts to guess the number.

Execution time- The time complexity is still the same because it allows a maximum of three attempts, and the invalid input does not change the maximum number of attempts.

Space complexity: The space complexity from the previous code is maintained, but an additional variable 'c' is added, so I did not improve the space complexity; the variable 'c' does not affect it.

Maintainability: I created "MAX_ATTEMPTS" and made "SECRET" a constant so these values can be easily changed. I also added comments throughout the code to make each part easier to understand and maintain. I also made sure that even with invalid input, the user gets three attempts to guess the number.